



VRIJE
UNIVERSITEIT
BRUSSEL



PHARO PROJECT

Metaprogramming and Reflection

Gérard Lichtert
0557513
gerard.lichtert@vub.be

May 2, 2026

Contents

1	Overview	2
2	Protected Visibility Modifier	2
2.1	Requirement 1	2
2.2	Requirement 2	3
3	Object-Centric Debugging	3
3.1	Requirement	3
4	Multiple exception handlers	3
4.1	Requirement 1	3
4.2	Requirement 2	3
4.3	Requirement 3	3
4.4	Requirement 4	4

1 Overview

This project consists of three parts.

- Protected Visibility Modifier
- Object-Centric Debugging
- Multiple exception handlers

Each part has its own .st file located in the /src directory. Do note, that I haven't overridden Pharo's default behavior. For instance, if you would like to use the protected Visibility Modifier, you will have to add the plugins to the compiler prior to testing it out. To make testing easier, there are already tests present that should cover the minimum implementation requirements, and test that the different component (such as plugins) indeed work as described.

You should be able to file in the .st files by dragging them in a Pharo 12.0 image, selecting "changelist browser", followed by "select all" and then "file in" selected. Note that if a class normally used in Pharo was moved due to an addition or modification of a method, a warning may pop up.

2 Protected Visibility Modifier

2.1 Requirement 1

The first requirement was to extend the compiler to support compilation of <protected> methods, using double registration of public methods and selector mangling of self sends.

To achieve this, first we created a plugin for the compiler that mangles the selector of methods that have the <protected> pragma. We also need double registration, to achieve this, I traced the method install to the point where methods are installed in a class. This was in ClassDescription, which is moved to the package since the method to install methods was modified.

A method is installed with, among others, a selector and a compiledMethod, to which we can ask if it has a <protected> pragma. If it does, we know it will already have a mangled selector. In this case it should be installed normally. However, if it does not have the <protected> pragma, we know it is a public method and therefore has to be installed twice, once with the normal selector and once with the mangled selector. Since a selector is required to be passed along prior to installation, we install it once normally and install it once but with the mangled selector. Essentially we add another message send but with the mangled selector.

To mangle self and super send, we create another plugin, just like we already did for the selector mangling of protected methods. However, this time we will mangle the messages sent to self and super receivers.

2.2 Requirement 2

Another requirement is that the modifier should be backwards compatible, meaning that sometimes objects will already exist without the mangled selector, or taking the `<protected>` modifier into account. To overcome this, we need to be extra careful when mangling self and super sends. More concretely, the plugin will now check if the class (or superclass, depending on the receiver) has the mangled method. If it does, it will mangle it, otherwise it will leave it as is.

3 Object-Centric Debugging

3.1 Requirement

4 Multiple exception handlers

For this part of the project we are focussing on adding multiple exception handlers to `BlockClosure`. Following the code snippet in the assignment description, we add a method to `BlockClosure` to convert it to a `MyBlockClosure`.

4.1 Requirement 1

Once a `BlockClosure` is a `MyBlockClosure`, it unlocks methods relevant for this part of the project. It unlocks methods such as `message:with:`, which allows the user to register handlers for exception classes. It does this by keeping them as an association in a collection internally. Initially, it tried to find the closes matching recorded Exception class, and uses the associated handler.

4.2 Requirement 2

To call the handlers in order of specificity, we change the way that `MyBlockClosure` finds the handler. First we create a `SortedCollection` from the `OrderedCollection` with associations, sorted by hierarchy and filtered if the thrown Exception is a subclass of it as well. After this, it simply throws the first one (the one that is the most specific to the exception thrown).

4.3 Requirement 3

To prevent breaking of `on:do:`, we essentially need to allow `MyBlockClosure` to allow an exception pass through if no handler can be found. This is done by simply allowing the exception to pass, if no handler can be found.

4.4 Requirement 4

Finally, to allow calling the next handler in `MyBlockClosure`, we modify `MyBlockClosure`, to keep track of the sorted collection of handlers as well as the handler currently being used. When `value:` is sent, it sets these values and when `callNextHandler` is sent, the index is incremented and the next handler in the collection gets used. If there is no handler left, it defaults to the behavior described in 4.3